Q03: Why RL After Pre-Training? RLHF, DPO, GRPO

DLS 2026 — Module (Paradise)

October 2026

Learning objectives

After this module you should be able to (i) explain why preference optimization cannot be reached by more next-token prediction, (ii) describe the RLHF loop and the role of the KL anchor, (iii) derive the DPO objective from the RLHF optimum, and (iv) state what GRPO changes and what all three methods still get criticized for.

1 The problem pre-training cannot solve

Pre-training optimizes next-token likelihood on a corpus. The corpus demonstrates *how people write*, not *how an assistant should respond*: it contains flame wars, half-finished drafts, and questions answered wrongly. Naively sampling a pretrained model yields text that is corpus-plausible but not assistant-*good* — it may ignore instructions, produce harmful content, or answer in the wrong register. SFT (supervised fine-tuning on demonstration responses) moves the model toward the right *behavioral family*, but demonstrations only show one good answer per prompt and say nothing about *relative* quality. What is missing is a training signal defined over *comparisons*: response A is better than response B for this prompt. Likelihood training cannot express a preference between two full responses without labeling one of them as a target sequence; RL can, because reward is an arbitrary scalar function of the whole output [3, 2].

That is the honest one-line answer to "why RL?": **the objective (human preference over complete responses) is not a likelihood**. Everything after is machinery for optimizing that non-likelihood objective without destroying the model.

2 RLHF: reward model + policy optimization + anchor

The InstructGPT recipe [3]:

1. **Collect preferences:** humans rank candidate responses to the same prompt.
2. **Fit a reward model** $r_\phi(x, y)$ from pairwise preferences, typically via the Bradley–Terry likelihood: $P(y_w \succ y_l) = \sigma(r_\phi(x, y_w) - r_\phi(x, y_l))$.
3. **Optimize the policy** (the LLM) to maximize reward, with a penalty for drifting from a reference policy π_{ref}:

$$\max_{\pi} \; \mathbb{E}_{x\sim\mathcal{D},\, y\sim\pi(\cdot|x)}\big[r_\phi(x, y)\big] - \beta\, \mathrm{KL}\big(\pi(\cdot|x) \,\|\, \pi_{\text{ref}}(\cdot|x)\big). \tag{1}$$

PPO [5] is the usual optimizer: the LLM is the policy, tokens are actions, and a value network estimates advantages for clipped surrogate updates.

Why the KL anchor matters. The reward model is only an *approximation* of human judgment, valid near the distribution it was trained on. Without the anchor, the policy finds adversarial outputs where r_ϕ is high but humans disagree — *reward hacking* — and can collapse to a degenerate style. The KL term keeps the policy in the region where the reward model can be trusted; β encodes how far you are willing to go. It is a trust region, not a safety feature per se.

Constitutional AI replaces part of the human preference collection with AI feedback guided by explicit principles [1], scaling the labeling without scaling human annotators (and shifting the questions to which principles and whose).

3 DPO: the closed-form shortcut

Equation (1) has an analytic solution: the optimal policy is a Boltzmann reweighting of the reference, $\pi^*(y|x) \propto \pi_{\text{ref}}(y|x)\exp\big(r(x,y)/\beta\big)$. Solving this expression for the reward gives $r(x,y) = \beta \log \frac{\pi^*(y|x)}{\pi_{\text{ref}}(y|x)} + \beta \log Z(x)$. Substituting into the Bradley–Terry preference model, the intractable partition function $Z(x)$ *cancels between the pair*, leaving a pure classification loss on the policy itself:

$$\mathcal{L}_{\text{DPO}} = -\mathbb{E}\Big[\log \sigma\Big(\beta \log \tfrac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \tfrac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)}\Big)\Big]. \tag{2}$$

DPO [4] thus optimizes the same preference data toward the same objective family *without sampling from the policy during training and without a reward model at all* — the log-ratio in (2) *is* an implicit reward. The trade-offs: no reward model means no reusable, inspectable reward signal, and the absence of explicit sampling makes the method sensitive to distribution shift (the policy is never probed on its own outputs during training).

4 GRPO: group-relative advantages

PPO's value network is expensive at LLM scale: it doubles the served parameter count during training. GRPO (Group Relative Policy Optimization, introduced in DeepSeekMath [6]) drops it. For each prompt, sample a *group* of G responses, score them all with the reward, and use the group's own statistics as the baseline:

$$\hat{A}_i = \frac{r_i - \text{mean}(r_1, \ldots, r_G)}{\text{std}(r_1, \ldots, r_G)}. \tag{3}$$

The advantage of each response is measured against its siblings — no value network, no stored rollouts. This works especially well with *verifiable* rewards (unit tests pass/fail, exact answers) where a group of samples naturally contains both successes and failures. GRPO is the backbone of RL with verifiable rewards in recent reasoning-model training.

5 Honest limitations (all methods)

- **Reward hacking:** any learned r_ϕ has exploits; the KL anchor mitigates, does not eliminate.
- **Length bias:** humans (and reward models) favor longer answers; models learn verbosity without substance.
- **Evaluator gaming:** where the evaluator is another model, the policy learns its blind spots (LLM-as-judge biases, style effects).

- **Distribution shift:** DPO never samples its own outputs; PPO/GRPO do, at cost.
- **Preferences are not truth:** preference optimization aligns to *raters*, with everything that implies about whose preferences and which failure modes transfer.

The one-sentence summary

RL enters after pre-training because "better" is a comparison between whole responses, not a likelihood; RLHF optimizes it with a learned reward plus a KL trust region, DPO eliminates the machinery via the closed-form optimum, and GRPO replaces the value network with group-relative baselines — all three inheriting the same core risks: reward hacking, length bias, and optimizing for raters rather than truth.

References

[1] Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, Anna Chen, Anna Goldie, Azalia Mirhoseini, et al. Constitutional AI: Harmlessness from AI feedback. *arXiv preprint arXiv:2212.08073*, 2022.

[2] Paul F. Christiano, Jan Leike, Tom Brown, Miljan Martic, Shane Legg, and Dario Amodei. Deep reinforcement learning from human preferences. *Advances in Neural Information Processing Systems*, 2017.

[3] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 2022.

[4] Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D. Manning, Stefano Ermon, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. In *Advances in Neural Information Processing Systems*, 2023.

[5] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[6] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y.K. Li, Y. Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.